0155. Min Stack

ulty: Medium | Category: Stack | LeetCode: https://leetcode.com/problems/min-s

Problem

- Design a stack with push, pop, top and getMin, each running in O(1).

Intuition

- The minimum only changes on push/pop, so store "min so far" alongside each element.
- When we pop, the previous minimum is automatically restored from the element below.

Brute Force

- Scan the stack on getMin: O(n).

Optimal Approach

- Each stack entry is a pair (value, minAtThisLevel).
- Push(x): min = empty ? x : min(x, top.min).
- GetMin returns top.min; pop simply removes the pair.

Worked Example

```
push -2   -> [(-2,-2)]
push  0   -> [(-2,-2),(0,-2)]
push -3   -> [... ,(-3,-3)]   getMin = -3
pop       -> top=(0,-2)       top = 0, getMin = -2
```

Java Solution

```java
public class MinStack {
    private final Deque<int[]> st = new ArrayDeque<>();

    public void push(int val) {
        int min = st.isEmpty() ? val : Math.min(val, st.peek()[1]);
        st.push(new int[]{val, min});
    }

    public void pop() { st.pop(); }

    public int top() { return st.peek()[0]; }

    public int getMin() { return st.peek()[1]; }
}
```

Complexity

- Time: O(1) for every operation. Space: O(n).

Pitfalls

- A single "current min" variable fails after popping the min.
- A two-stack variant (push to minStack only when x <= min) must use <= to handle duplicates.